PREGRADO

UNIDAD 1 | OVERVIEW & DESARROLLO DE APIS ROBUSTAS CON FRAMEWORKS DE BACKEND

SPRING DATA JPA

ASI0705 | Arquitectura de Aplicaciones Web

Logro de la unidad: Al finalizar esta unidad, el estudiante será capaz de desarrollar APIs robustas utilizando el framework Spring Boot, implementando seguridad, integración con bases de datos, entendiendo el impacto de las APIs en los negocios.

AGENDA

INTRO

JPA REPOSITORIES

QUERY METHODS

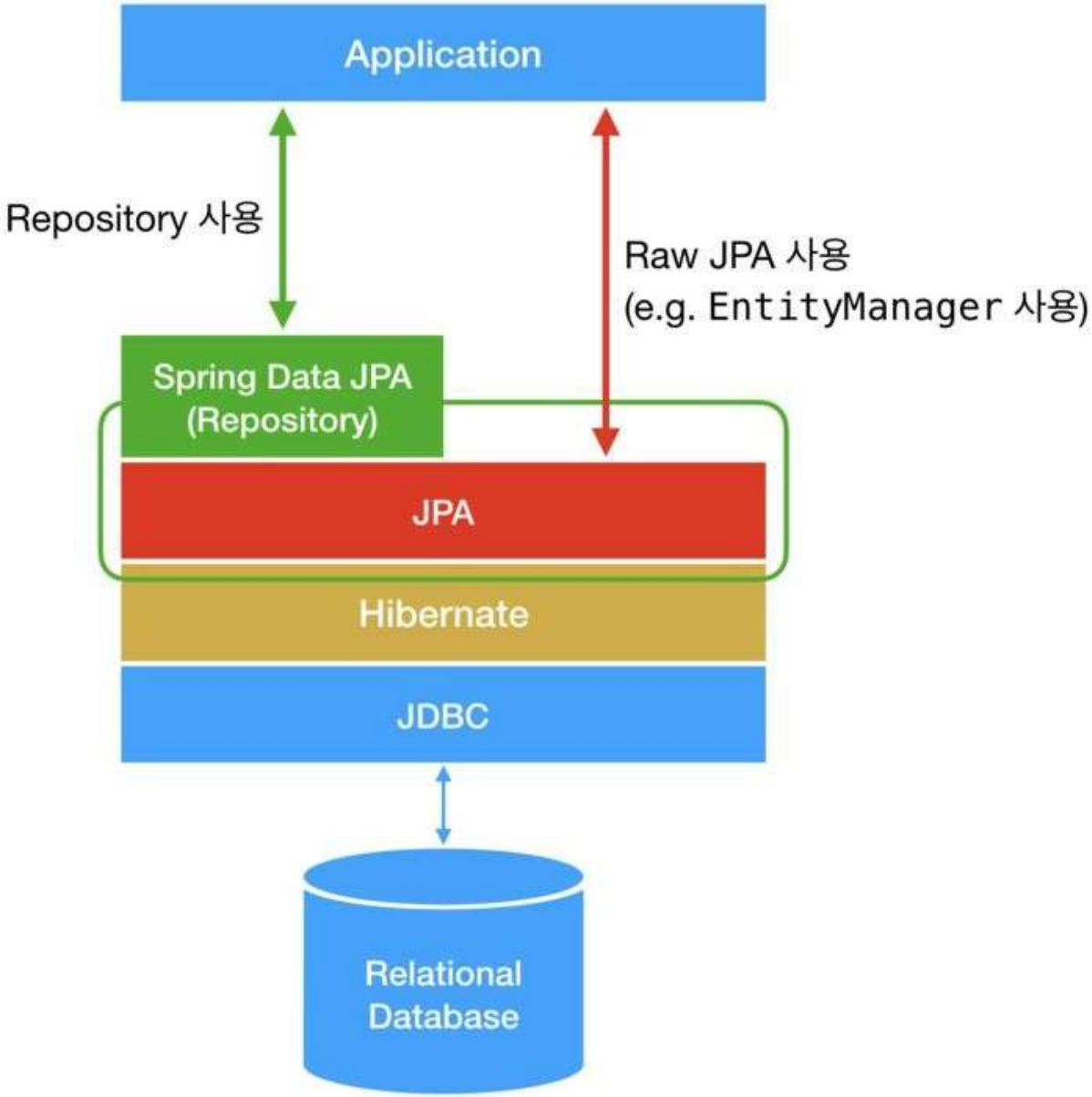
NAMED & NATIVE QUERIES

SPRING DATA JPA

¿Qué es Spring DataJPA?

- Spring Data JPA, parte de la familia Spring Data más grande, facilita la implementación de repositorios basados en JPA.
- Este módulo trata sobre el soporte mejorado para las capas de acceso a datos basadas en JPA.
- Facilita la creación de aplicaciones impulsadas por Spring que utilizan tecnologías de acceso a datos.

¿Qué es Spring DataJPA?

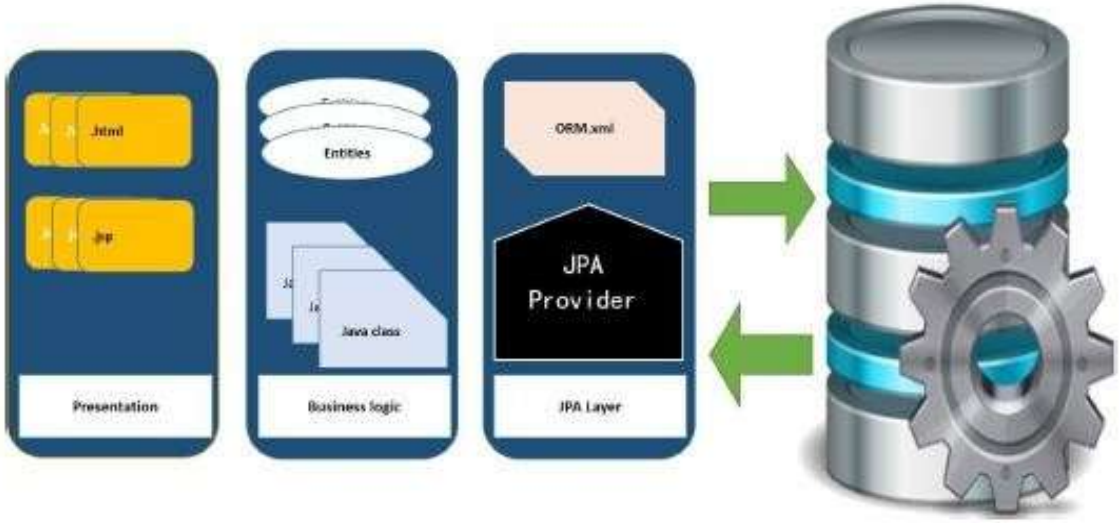


Características de Spring DataJPA

- Soporte sofisticado para construir repositorios basados en Spring y JPA
- Compatibilidad con predicados Querydsl y, por lo tanto, consultas JPA con seguridad de tipos
- Compatibilidad con paginación, ejecución dinámica de consultas, capacidad para integrar código de acceso a datos personalizado
- Validación de @Query anotadas en el momento del arranque
- Configuración de repositorio basada en JavaConfig introduciendo @EnableJpaRepositories.
- Soporte sofisticado para construir repositories basados en Spring y JPA
- Validación de @Query consultas anotadas en el momento de arranque
- Soporte de paginación, ejecución de consultas dinámicas, capacidad de integrar código de acceso a datos personalizado.

Java Persistence API (JPA)

Es un conjunto de clases y métodos que permiten almacenar datos en una base de datos, estableciendo una correlación entre una base de datos relacional y un sistema orientado a objetos.

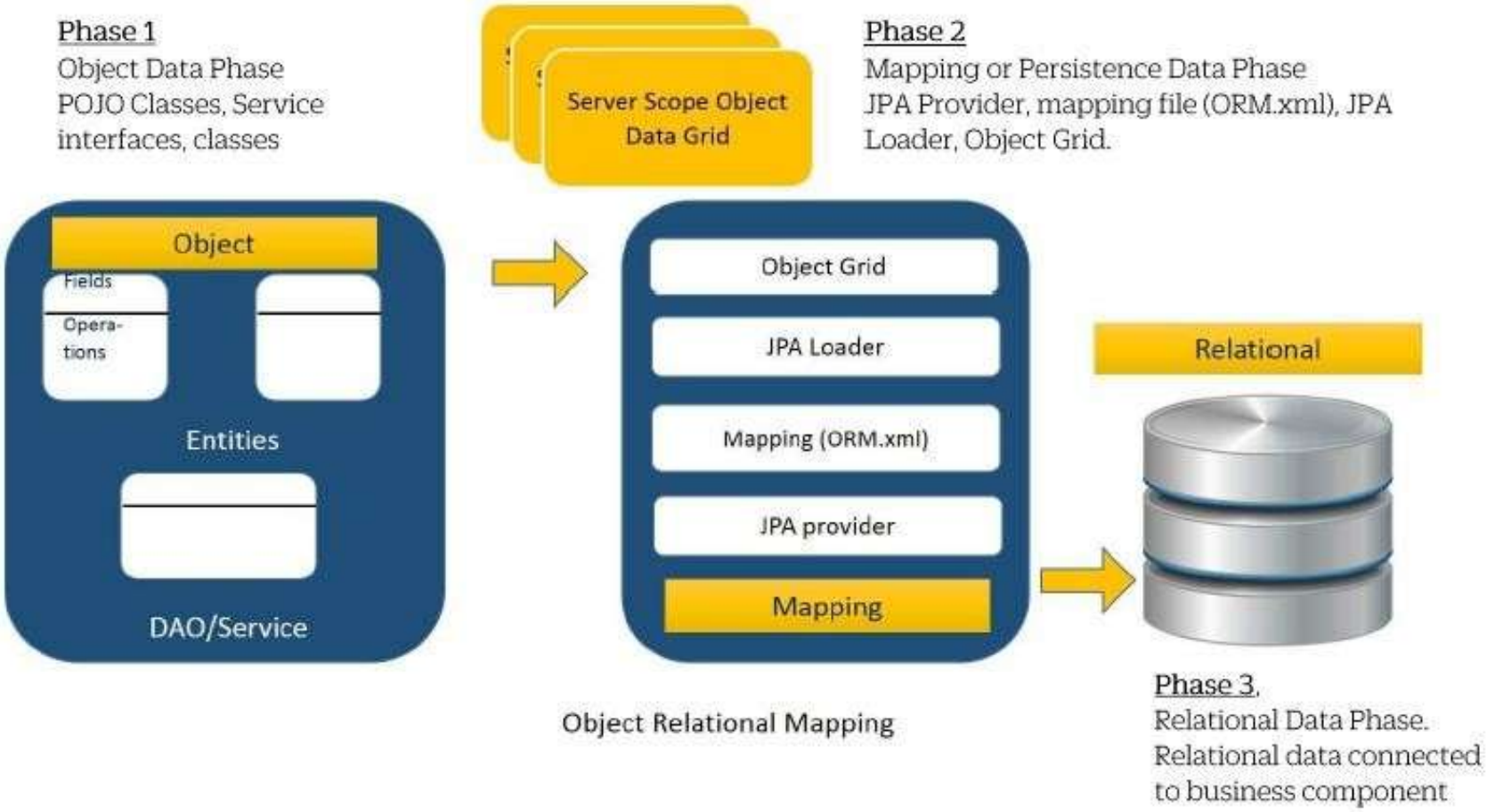


¿Dónde usar?

Para reducir la carga de escribir códigos relacionales para gestión de objetos, un programador sigue el "Proveedor" marco JPA, que permite la fácil interacción con instancia de la base de datos.

JPA & Object Relational Mapping

La implementación de Object Relational Mapping (ORM) que realiza JPA considera tres fases principales: Object Data Phase, Mapping or Persistence Phase, Relational Data Phase.



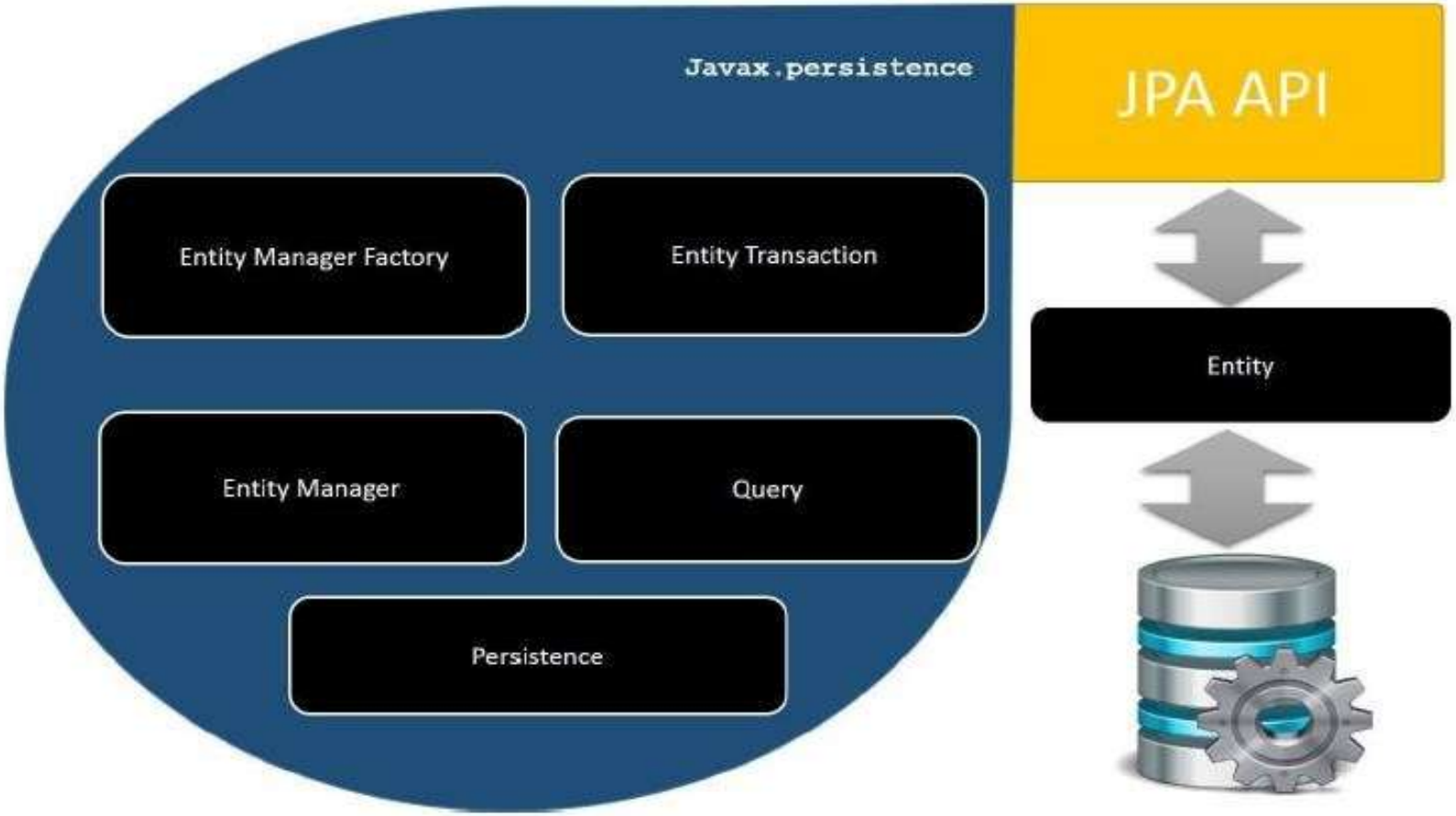
JPA Providers

JPA establece una interfaz común que es implementada por proveedores de persistencia de nuestra elección :

- Hibernate
- Eclipselink
- Toplink
- Spring Data JPA
- etc

JPA Architecture

Clases principales y las interfaces de JPA

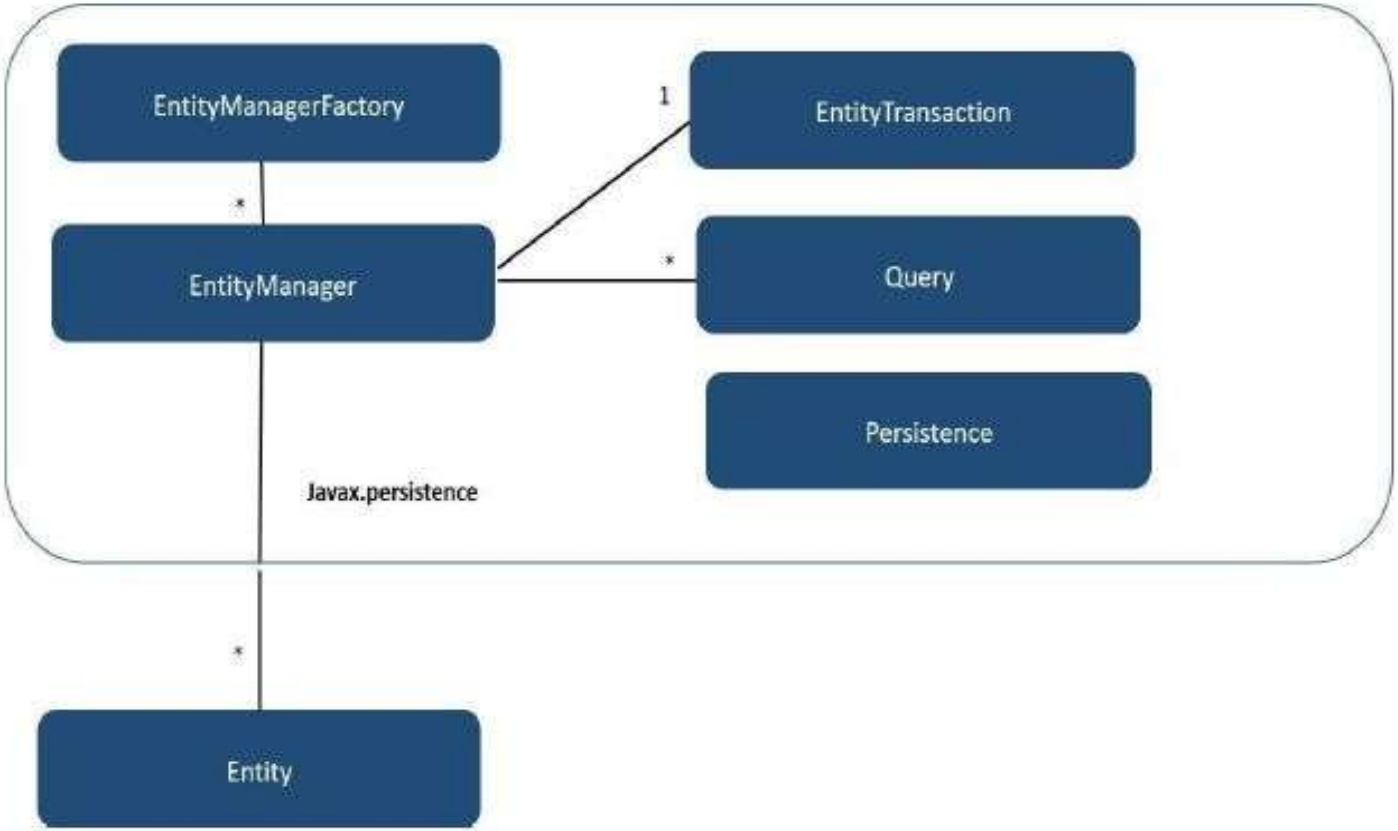


JPA Architecture

Unidades	Descripción
EntityManagerFactory	Esta es un factory class para EntityManager. Crea y gestiona múltiples instancias EntityManager.
EntityManager.	Es una interfaz, que gestiona la persistencia de objetos. Funciona como un factory para instancias de Query.
Entity	Las entidades son los objetos de persistencia, almacenados como registros en la base de datos.
EntityTransaction	Tiene una relación de uno a uno con EntityManager. Para cada EntityManager, sus operaciones son mantenidas por la clase EntityTransaction.
Persistence	Esta clase contiene métodos estáticos para obtener instancias de EntityManagerFactory.
Query	Esta interfaz es implementada por cada JPA provider para obtener objetos relacionales que cumplan con los criterios.

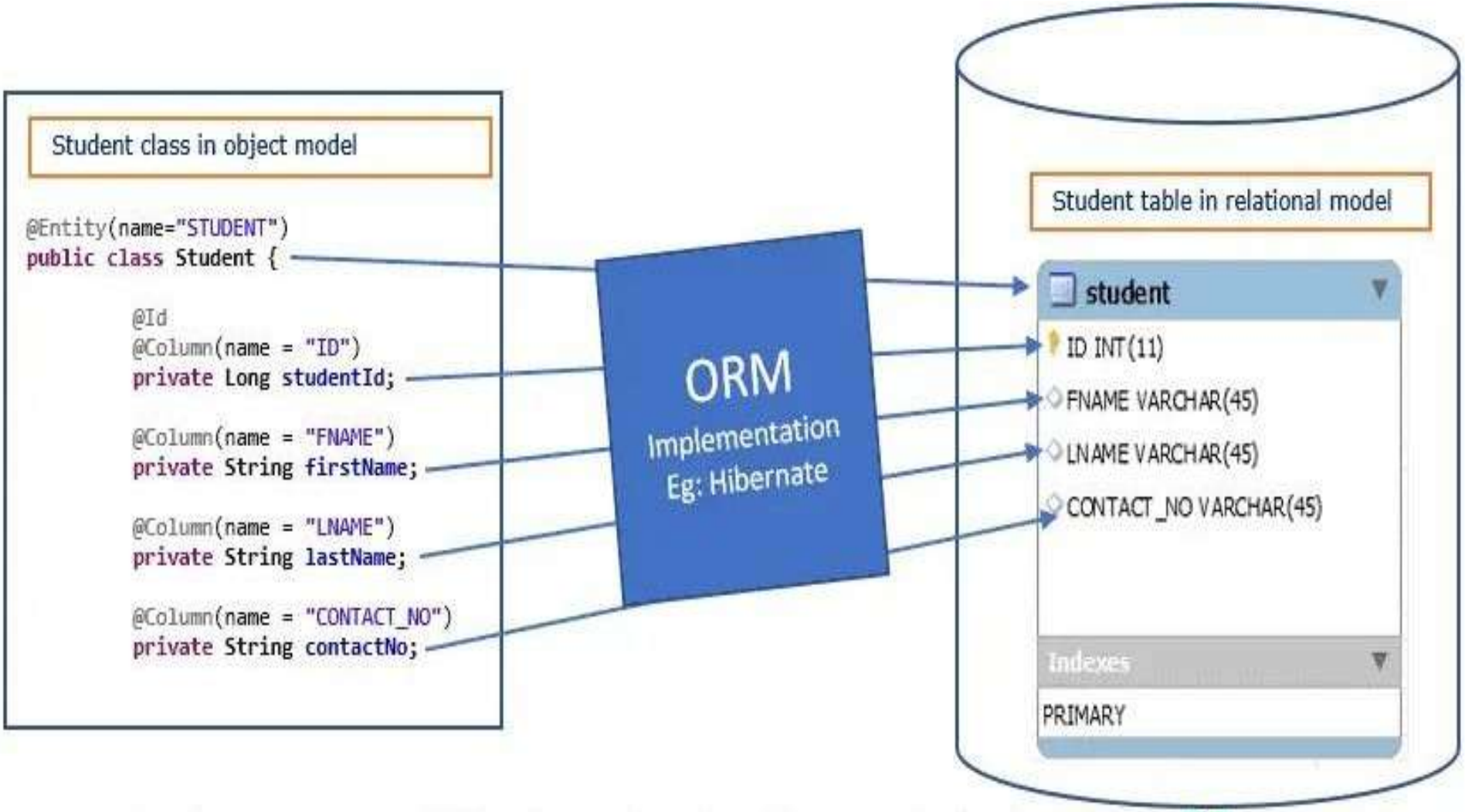
Las relaciones entre las clases JPA

javax.persistence package*



* En migración a jakarta.persistence

ORM



ORM implements responsibility of mapping the Object to Relational Model.

JavaByDeveloper

Definir una entidad simple

En este ejemplo, almacena Customer objetos, cada uno anotado como una entidad JPA. El siguiente listado muestra la clase Cliente (en src/main/java/com/example/accessingdatajpa/Customer.java):

```
package com.example.accessingdatajpa;

import javax.persistence.Entity;
import  javax.persistence.GeneratedValue;
import   javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Customer {

  @Id
  @GeneratedValue(strategy=GenerationType.AUTO)
  private Long id;
  private String firstName;
  private String lastName;

  protected Customer() {}

  public Customer(String firstName, String lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
  }

  @Override
  public String toString() {
    return String.format(
        "Customer[id=%d, firstName='%s', lastName='%s']",
        id, firstName, lastName);
  }
  public Long getId() {
    return id;
  }
  public String getFirstName() {
    return firstName;
  }
  public String getLastName() {
    return lastName;
  }
}
```

En la clase **Customer**, tiene tres atributos: id, firstName y lastName. También tienes dos constructores. El constructor predeterminado existe solo por el bien de JPA. No lo usa directamente, por lo que se designa como protected. El otro constructor es el que usa para crear instancias de Customer para guardar en la base de datos.

La clase Customer se anota con @Entity, lo que indica que es una entidad JPA. (Debido a que no @Tableexiste ninguna anotación, se supone que esta entidad está asignada a una tabla denominada Customer).

La propiedad id de la clase Customer se anota con @Id para que JPA la reconozca como la ID del objeto. La id también se anota con @GeneratedValue para indicar que el ID debe generarse automáticamente.

Las otras dos propiedades firstName y lastName, se dejan sin anotar. Se supone que se asignan a columnas que comparten los mismos nombres que las propias propiedades.

El método conveniente toString() imprime las propiedades del cliente.

Annotations

- En la definición de la clase, podemos escribir la configuración con anotaciones.
- Las anotaciones se utilizan para las clases, propiedades y métodos.
- Las anotaciones comienzan con "@" el símbolo.
- Las anotaciones son declaradas antes de una clase, propiedad o método.
- Todas las anotaciones de JPA se definen en el **javax.persistence** paquete.

Annotation	Descripción
@Entity	Declara la clase como una entidad o una tabla.
@Table	Declara nombre de la tabla.
@Basic	Especifica de forma explícita no constraint fields.
@Embedded	Especifica las propiedades de la clase o de una entidad cuyo valor es una instancia de una embeddable class.
@Id	Especifica la propiedad usada para identificar la clase (la clave principal de una tabla).

Annotations

Annotation	Descripción
@GeneratedValue	Especifica el modo en que el identity attibute se puede incializar como automática, manual o con un valor tomado de la tabla de secuencias.
@Transient	Especifica la propiedad que no es persistente, es decirque no se almacena en la base de datos.
@Column	Utiliza para especificar un atributo como una columna o propiedad persistente.
@SequenceGenerator	Utilizada para definir el valor de la propiedad que se especifica en la anotación @GeneratedValue. Crea una sequence.
@TableGenerator	Utilizada para definir el valor de la propiedad que se especifica en la anotación @GeneratedValue. Crea una tabla para generación del valor.
@JoinColumn	Especifica entity association o entity collection. Esto se utiliza en asociaciones de tipo many-to-one y one-to-many.

Annotations

Anotación	Descripción
@UniqueConstraint	Especifica los campos y las restricciones unique para la primaria o la secundaria.
@ManyToMany	Define una relación many-to-many entre tables.
@ManyToOne	Define una relación de many-to-one entre tables.
@OneToMany	Define una relación one-to-many entre tables.
@OneToOne	Define una relación one-to-one entre tables.
@NamedQueries	Especifica la lista de Named Queries.
@NamedQuery	Especifica una Named Query usando un nombre estático.

JPQL

Es un lenguaje de consulta de entidades/objetos en lugar de tablas y registros. El lenguaje nos proporciona una manera de expresar consultas en términos de entidades y sus relaciones operando sobre las entidades en lugar de operar sobre las tablas de la base de datos.

Tipos de Sentencias:

SENTENCIAS SELECT
SENTENCIAS AGGREGATE
SENTENCIAS UPDATE
SENTENCIAS DELETE

Ejemplos de sentencias JPQL

```
SELECT e FROM Employee e
```

```
SELECT e.name, e.salary FROM Employee e
```

Inner Join

```
SELECT p FROM Employee e JOIN e.phones p
```

Su traducción a SQL será:

```
SELECT p.id, p.phone_num, p.type, p.emp_id
FROM emp e, phone p WHERE e.id = p.emp_id
```

Ejemplos de sentencias JPQL

```
SELECT e
FROM Employee e
WHERE e.salary
BETWEEN 40000 AND 45000
```

```
SELECT d FROM Department d
WHERE d.name
LIKE '    Eng%'
```

```
SELECT e
FROM Employee e
WHERE e.address.state
IN ('NY', 'CA')
```

Ejemplos de sentencias JPQL

```
SELECT e, d
FROM Employee e
JOIN e.department d
ORDER BY d.name, e.name DESC
```

```
SELECT d.name, COUNT(e), AVG(e.salary)
FROM Department d JOIN d.employees e
GROUP BY d.name
```

```
UPDATE Phone p
SET p.number = ‘4356’
WHERE p.employee.address.city = 'Ottawa'
AND p.type = 'Office'
```

```
DELETE FROM Employee e
WHERE e.department IS NULL
```

AGENDA

INTRO

JPA REPOSITORIES

QUERY METHODS

NAMED & NATIVE QUERIES

Spring Data JPA Repositories

Los repositorios Spring estándar y los repositorios Spring Data JPA difieren ligeramente en concepto y estructura.

Aquí están las principales diferencias:

- Interfaz Java en lugar de una clase
- Mapa 1 a 1 con una entidad JPA
- Centrarse en el contrato DAO

Todos los repositorios JPA son interfaces Java en lugar de clases. Estas interfaces están asociadas con una entidad JPA. Cada repositorio JPA solo puede realizar operaciones de acceso a datos para esa entidad en particular y sus atributos de datos. Esto ayuda a enfocar el repositorio JPA en el contrato DAO para esa entidad y sus datos de respaldo.

Spring Data JPA Repositories

Al suministrar la entidad JPA y su tipo de datos de clave principal, el repositorio JPA ahora sabe exactamente con qué tabla de base de datos en columnas puede trabajar porque toda esa información está bien agrupada dentro de su entidad JPA.

```
public interface MyJpaRepository
    extends JpaRepository<Entity, Id Type> {}
```

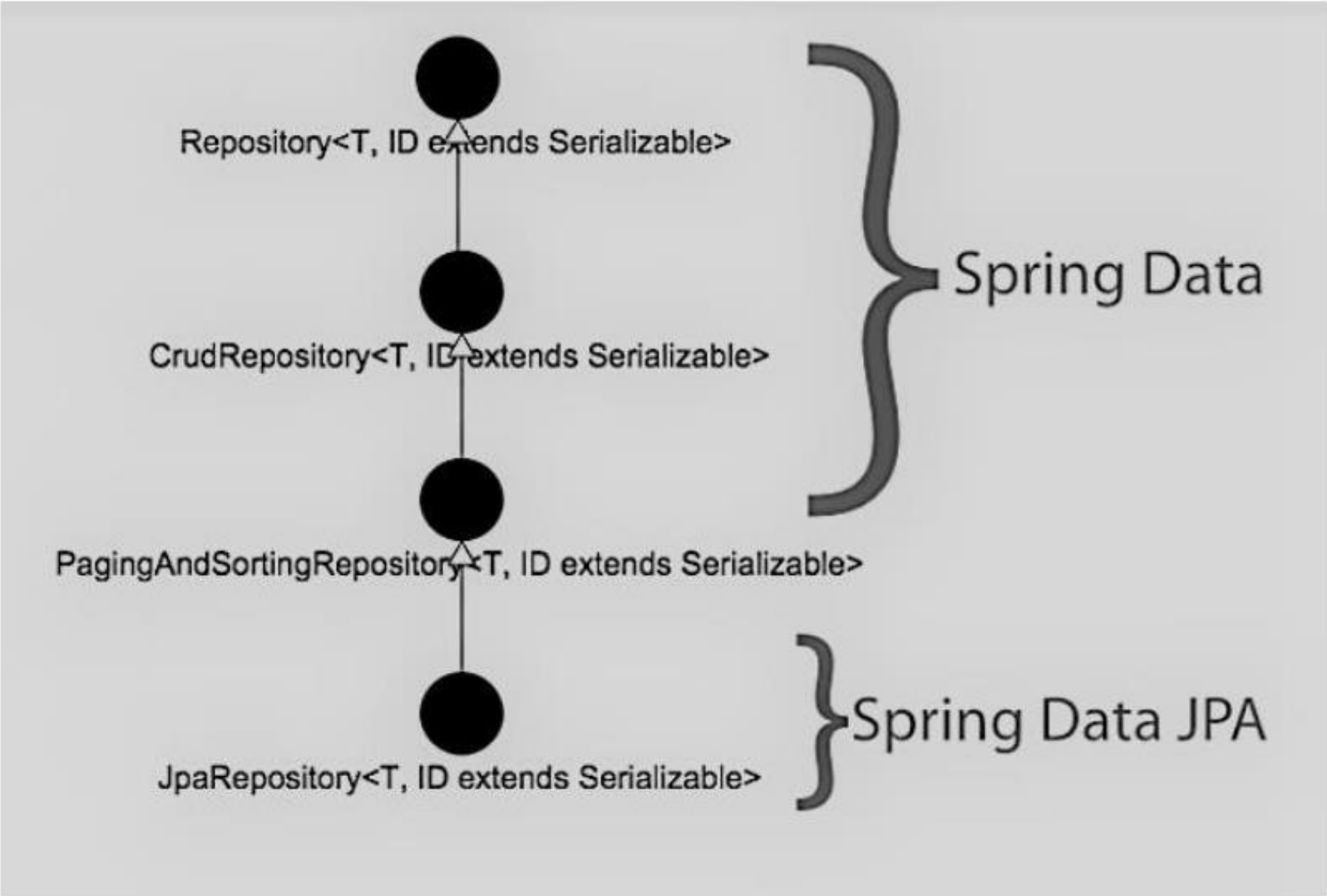
El patrón DAO nos permite como desarrolladores enfocarnos en el contrato DAO mientras cumplimos el objetivo de Spring Data JPA de simplificar nuestra capa de acceso a datos sin ninguna pérdida de funcionalidad.

JpaRepository Features

Cuando extiende la interfaz del repositorio JPA, también obtiene acceso a muchas otras características. La funcionalidad que viene con el repositorio JPA incluye las operaciones CRUD.

Funcionalidad:

- Query DSL
- CRUD operations
- Paging and sorting
- Helpers
  - count()
  - exists(Long id)
  - flush()
  - deleteInBatch(Iterable entities)


Repository<T, ID extends Serializable>
CrudRepository<T, ID extends Serializable>
PagingAndSortingRepository<T, ID extends Serializable>
JpaRepository<T, ID extends Serializable>
Spring Data
Spring Data JPA

AGENDA

INTRO

JPA REPOSITORIES

QUERY METHODS

NAMED & NATIVE QUERIES

Query Methods

El módulo JPA admite la definición manual de una consulta como String o se deriva del nombre del método.
Creación de Consultas a partir de Nombres de Métodos: El método tendrá el prefijo **findBy** y contendrá el nombre de los atributos por los que se quiera filtrar unidos por los operadores **And**, **Or**, **Between**, **LessThan**, **GreaterThan**.

```
public interface EmployeeRepository extends CrudRepository<Employee, Long> {
  List<Employee> findByDept(String deptName);
  List<Employee> findBySalaryGreaterThan(int salary);
  List<Employee> findByDeptAndSalaryLessThan(String deptName, int salary);
}
```

Query Methods

Además de **findBy** se pueden usar el prefijo **countBy** para devolver el número de objetos que verifican los filtros definidos:

```
public interface EmployeeRepository extends CrudRepository<Employee, Long> {
  long countByDept(String deptName);
  long countBySalaryGreaterThanEqual(int salary);
  long countByNameEndingWith(String endString);
  long countByNameLike(String likeString);
}
```

Query Method Supported Keywords

Keyword	Sample	JPQL snippet
And	findByLastnameAndFirstname	... where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	... where x.lastname = ?1 or x.firstname = ?2
Is, Equals	findByFirstname,findByFirstnameIs,findByFirstnameEquals	... where x.firstname = ?1
Between	findByStartDateBetween	... where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	... where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	... where x.age <= ?1
GreaterThan	findByAgeGreaterThan	... where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	... where x.age >= ?1
After	findByStartDateAfter	... where x.startDate > ?1
Before	findByStartDateBefore	... where x.startDate < ?1
IsNull, Null	findByAge(Is)Null	... where x.age is null
IsNotNull, NotNull	findByAge(Is)NotNull	... where x.age not null
Like	findByFirstnameLike	... where x.firstname like ?1
NotLike	findByFirstnameNotLike	... where x.firstname not like ?1
StartingWith	findByFirstnameStartingWith	... where x.firstname like ?1 (parameter bound with appended %)
EndingWith	findByFirstnameEndingWith	... where x.firstname like ?1 (parameter bound with prepended %)
Containing	findByFirstnameContaining	... where x.firstname like ?1 (parameter bound wrapped in %)
OrderBy	findByAgeOrderByLastnameDesc	... where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot	... where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	... where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	... where x.age not in ?1
True	findByActiveTrue()	... where x.active = true
False	findByActiveFalse()	... where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	... where UPPER(x.firstame) = UPPER(?1)

AGENDA

INTRO

JPA REPOSITORIES

QUERY METHODS

NAMED & NATIVE QUERIES

Named Queries

Puede utilizarse @Query

```
public interface UserRepository extends JpaRepository<User, Long> {


  @Query("select u from User u where u.emailAddress = ?1")
  User findByEmailAddress(String emailAddress);

  @Query("select u from User u where u.firstname like %?1")
  List<User> findByFirstnameEndsWith(String firstname);

}
```

Named Parameters

Puede utilizarse @Param

```
public interface UserRepository extends JpaRepository<User, Long> {

  @Query("select u from User u where u.firstName = :firstName or u.lastName = :lastName")
  User findByLastNameOrFirstName(@Param("lastName") String lastName,
                                 @Param("firstName") String firstName);
}
```

Native Queries

Puede utilizars el flag **nativeQuery** de @Query

```
public interface UserRepository extends JpaRepository<User, Long> {

  @Query(value = "SELECT * FROM users WHERE email_address = ?1", nativeQuery = true)
  User findByEmailAddress(String emailAddress);
}
```

Sort

Puede utilizarse **Sort** o **JpaSort**

```
public interface UserRepository extends JpaRepository<User, Long> {

  @Query("select u from User u where u.lastName like ?1%")
  List<User> findByAndSort(String lastName, Sort sort);

  @Query("select u.id, LENGTH(u.firstName) as fn_len from User u where u.lastName like ?1%")
  List<Object[]> findByAsArrayAndSort(String lastName, Sort sort);
}
```

```
repo.findByAndSort("lannister", new Sort("firstName"));
repo.findByAndSort("stark", new Sort("LENGTH(firstName)")); // Invalid, throws Exception
repo.findByAndSort("targaryen", JpaSort.unsafe("LENGTH(firstName)"));
repo.findByAsArrayAndSort("bolton", new Sort("fn_len"));
```

Logging

La anotación @Slf4j en Spring Boot es una anotación de Lombok que permite integrar SLF4J (Simple Logging Facade for Java) en una clase sin necesidad de declarar manualmente un objeto Logger.

Para qué se usa @Slf4j?

- Facilita el uso de logs sin necesidad de crear una instancia de Logger manualmente.
- Permite imprimir mensajes de depuración e información en consola o en un archivo de log.
- Optimiza el código al evitar la declaración explícita de LoggerFactory.getLogger().

Ejemplo de uso de @Slf4j

```java
@RestController
@RequestMapping("/productos")
@Slf4j  // <-- Habilita el uso de logs sin declarar un Logger manualmente
public class ProductoController {
    private final ProductoService productoService;

    public ProductoController(ProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping
    public List<Producto> obtenerProductos() {
        log.info("Solicitud GET para obtener productos");
        return productoService.obtenerProductos();
    }

    @PostMapping
    public Producto guardarProducto(@RequestBody Producto producto) {
        log.info("Solicitud POST para guardar producto: {}", producto.getNom
        return productoService.guardarProducto(producto);
    }

    @DeleteMapping("/{id}")
    public void eliminarProducto(@PathVariable long id) {
        log.warn("Solicitud DELETE para elimi producto con ID: {}", id);
        productoService.eliminarProducto(id);
```

Niveles de Logs disponibles con @Slf4j

```
log.trace("Mensaje TRACE - Depuración detallada");
log.debug("Mensaje DEBUG - Información útil para desarrollo");
log.info("Mensaje INFO - Información general del sistema");
log.warn("Mensaje WARN - Advertencia de posible problema");
log.error("Mensaje ERROR - Error en el sistema");
```

Ejemplo de salida en consola:

```
2024-03-03 12:00:00 - [INFO] - Solicitud GET para obtener productos
2024-03-03 12:00:02 - [WARN] - Solicitud DELETE para eliminar producto con ID: 5
2024-03-03 12:00:05 - [ERROR] - No se encontró el producto con ID: 10
```

REFERENCIAS

Para profundizar

https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#reference

https://stackabuse.com/guide-to-spring-data-jpa/

PREGRADO

Ingeniería de Sistemas de Información
Escuela de Ingeniería de Sistemas y Computación | Facultad de Ingeniería

UPC
Universidad Peruana
de Ciencias Aplicadas

Prolongación Primavera 2390,
Monterrico, Santiago de Surco
Lima 33 - Perú
T 511 313 3333
https://www.upc.edu.pe

exígete, innova

LAUREATE
INTERNATIONAL
UNIVERSITIES